Huyin Song (25331396) ENGE707 Kaggle AI Competition Assessment

Kaggle username: huyinsong (Huyin Song)

TASK 1:

This problem is a case of image recognition and classification. The idea is to construct, evaluate, and optimize a machine learning pipeline that when given a bunch of images the machine will be able to classify each image into their likely respective classes/types of the 8 named; CatEye, DogEye, CatFace, DogFace, CatMouth, DogMouth, CatNose, or DogNose. For example, give the machine ten images of a cat's eye. The optimal result would be the machine correctly classifying all ten of those images as 'CatEye'.

I start off by inspecting the structure of the training and test data through Python and Microsoft Excel. There are 63,660 jpg files present inside the training folder and 29,443 jpg files inside the test folder. Simple observation shows nearly all images are of cats and dogs and their various parts like their eyes, nose, and mouth. The given excel file has just two features, 'image_id' and 'class', containing the image index ID associated with the training folder's images and class as the correct class that each image should be of. 'image_id' is a 64-bit integer variable while 'class' is a string. This means I will need to encode the class feature into a numerical variable in order for the machine to feasibly use it.

With a simple python cell, I find the number of images in each class, they are;

Class	*Image count*	*Percentage (%)*
CatEye	13,884	21.81
CatFace	6,942	10.90
CatMouth	6,942	10.90
CatNose	6,942	10.90
DogEye	11,580	18.19
DogFace	5,790	9.10
DogMouth	5,790	9.10
DogNose	5,790	9.10

So, it seems we have a class representation imbalance. CatEye and DogEye are easily the most represented class out of all classes, skewing representation in their favour when compared to smaller classes like DogNose and such. This will need to be address.

Next, representative sample images were generated with Python of each class and their corresponding image. For simplicity I decided to use the very first image of each class be their representative. Please see the image below:



I performed both manual and code checks to inspect the images and their properties. Manual review reveals significant differences throughout image resolutions; this will be address later. With code it was also revealed that there were no corrupted images as far as I am aware (Python checked and could read every image fine), there were unfortunately duplicate images present (for example, training images 00356.jpg and 06903.jpg are exactly identical). To preserve the complete database and training files I decided it would be best to only remove the duplicates and their associated labels in the database in only local memory in each Kernel, thus I will do so by code, there are only 15 duplicates so the training integrity should remain okay. Checking blurriness in an image is a bit of a pain so I opted for a surface level manual check, I generally found most images to be not blurry but did find one image 07413.jpg to be a bit blurry, we can conclude blurriness is present but roughly not a majority nor a significant portion of the population (Please see the image below for image 07413.jpg). As for unclear class boundaries a surface level manual check was done, images looked good did not find an unclear one, cannot conclude all images are okay but automatic tests in Python is tricky.



My plan is to split the training images into 80/20, 80% of the images and labels to train and 20% to test to evaluate the performance on how well it did. I may then optimize the model for accuracy as that the is Kaggle competition's measured metric, then I will use the model on the test image set to generate a corresponding submission database to submit.

TASK 2:

To address the variance in image sizes and resolutions, I have decided to shrink or scale larger or smaller images to 128 x 128 pixels. I handled this with the transforms Resize and ToTensor functions which should be hopefully sufficient. Furthermore, Transforms also gave me the ability to implement a RandomHorizontalFlip and RandomRotation to which I set to 10 degrees and p=0.5, which should sometimes introduce more variance into the training data to better generalize the model. I specifically did this in the local memory; the original image set is untouched and is only temporary in each Kernel run. The idea is for the model to get better acquainted with images that aren't perfectly straight, the more training exposure the model gets with slightly rotated images the better it generalises.

Next, I mapped the either different string labels to integer values from 0 to 7 which logically is sufficient and would not cause errors. They are clear and distinct.

To ensure efficient data loading as we have quite a number of records and images, I used a dataloader objects with a batch size of 64 giving me 796 training batches as well as 199 validation batches after the 80/20 training split (seed of 42 to maintain reproducibility). This should keep the data loading controllable and efficient.

As for avoiding data leakage, I have tried to keep it contained and are unaware of any leakages, I will update this report if I find any.

Update: I found a small leakage where I specified that I had removed the duplicates from the local memory database of the kernel, but later on in the code I had not. The effect is negligible for such a small number of images but should try to have it removed if I can.

TASK 3:

I chose to use a custom CNN architecture model for this application. I reasoned it was suitable because CNNs were designed to handle digital images in image classification which is exactly what I need. Furthermore, it handles computational resource efficiently and can be shaped to match whatever custom needs I need (cat and dog classification for example).

To briefly describe the architecture and components; I pass it the 128 x 128 pixel RGB image of what I need classified in the Input layer, the convolutional layer is next up and detects basic patterns in the image such as the colours, edges, and textures. Next it goes to the Activation ReLU function where CNN learns complex patterns of the image that couldn't be captured with just the Convolutional layer. Next is the Pooling layer where we apply Max Pooling, basically reducing the special dimensions while keeping the important responses. Next the data is passed onto the flatten layer where it is compressed into a single value where it then goes into the Fully Connected layer where it uses the value and extracted features to make a prediction. Lastly, out comes the prediction value which we can use as the machine's most likely guess at what exactly classification the image provided is of. The architecture is quite linear and has few components.

The output image; 8 output units on the 8 target classes as I've described in task 1, with ReLU as the activation and prioritizes the index with the highest output score.

As for the various settings, I chose to use a loss function that penalises the model for incorrect class predictions and to apply heavier weights to underrepresented classes as we have a class imbalance present (the weights of which I calculated and now am applying). This should encourage the model to try to improve its accuracy while addressing the class imbalance. I used an 'Adam' optimiser with a 0.001 learning rate, I reasoned Adam being the popular widely used starting option should be a decent pick for my application, then choose 0.001 learning rate to be a starter baseline. These choices may not be optimal and are simply my arbitrary choice.

To run the first training, I again used the 64 batch size which shouldn't be too large nor small paired with a 10 epoch count. Again, not too little not too much I reason. Training took roughly 10 minutes on my RTX 3070 with CUDA, epoch 9 had the highest accuracy metric at 74.91% validation accuracy, to which I restored the checkpoint and will use. In theory, this pipeline should be capable of generating the Kaggle predictions I need for submission.

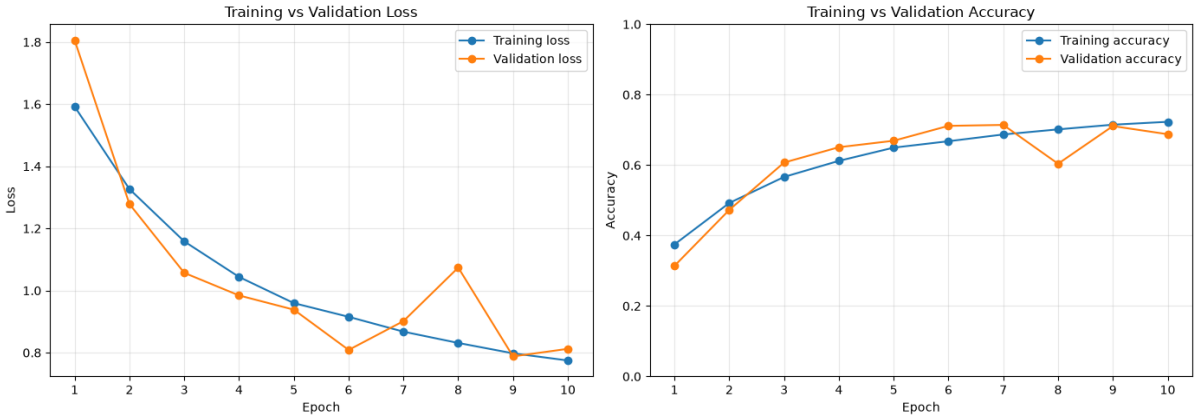
Task 4:

Several strategies have already been applied to help improve the generalisation of the model during training. For example, I applied data augmentation when I applied the Transforms function giving some images a slight rotation or a random horizontal flip, this helps the model train on images that aren't 'well-oriented'. I used a dropout of p=0.3, as well as something like best-checkpoint selection where I reloaded epoch 9 which had the best measured accuracy of all 10 epochs. These

techniques should boost the generalisation of the model and help it perform more accurately when benchmarked on the test cases.

Given that the metric being measured in the Kaggle competition is accuracy, I believe I should refine my training strategy a little towards aiming for the highest accuracy I could possibly get even at the cost of the other performance metrics and measures.

I generated some graphs on training vs validation both loss and accuracy, please see below.



The graph shows a generally good fit, no significant evidence of persistent underfitting or overfitting.

Epochs 1 to 5 in validation loss shows the model learning quite substantially in the useful features of the images and does show large improvements in its predictions. Validation loss epoch 6 hits a large dip in loss but jumps up at epoch 8 before dropping down at epoch 9 and 10. The signs points towards a temporary disturbance in validation loss as opposed to a sustained and persistent pattern, I believe this is okay as is.

As for validation accuracy, it sharply rises between epochs 1 to 3 then slows down between epochs 4 to 7 with a slow but steady increase. Epoch 8 drops validation accuracy down from a score of roughly 70% to 60%, before quacking rising back to 70% for epochs 9 and falls slightly to epoch 10. The sudden change does suggest some level of instability in generalisation, but the overall level of stabilisation is encouraging in my opinion.

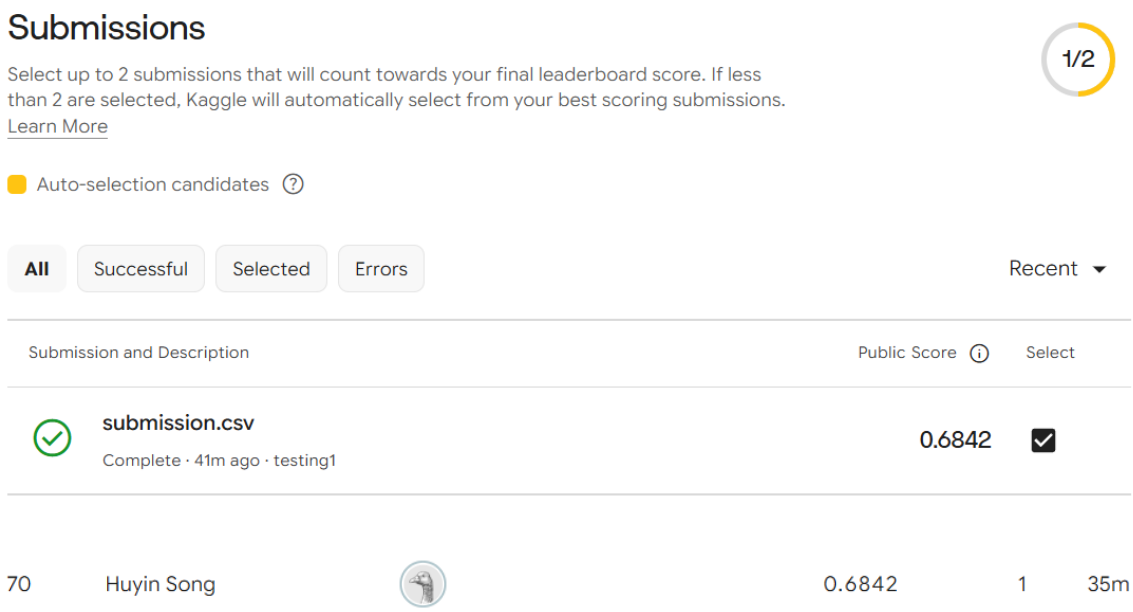
One pattern to note is the variances in both validation loss and validation accuracy are both generally occurring towards the later epochs.

Task 5:

I have generated a 'submission.csv' file to upload and benchmark on Kaggle.

It is composed of 29,443 records with two columns/features. Feature 1 is 'image_id' that pairs the corresponding image in the image test folder to feature 2. Feature is 'class' where it stores my model's best guess at which class of the target classes that the corresponding image is.

The result came back with a 68.42% score which at the time of writing places me at 70th place. I need to optimize for accuracy which I have not yet done.



Top 10 scores show a plateau in accuracy metric around 98% never reaching 99%, I should aim for at least 95% or higher to achieve a 'good result'.

At the moment, it is far from a decent result and holds last place position. I plan to sacrifice all other metrics to optimize purely on accuracy as it is the only metric that is measured and matters in this exact context.

I evaluated the performance of this model as it is currently with the various metrics. I ran evaluation on 12,732 images within the training 80/20 split.

```
Validation loss: 0.7318
Validation accuracy: 74.9057%

Classification report:
              precision    recall  f1-score   support

      CatEye     0.9347    0.7731    0.8463      2777
     CatFace     0.9413    0.8775    0.9083      1388
    CatMouth     0.6408    0.8999    0.7486      1388
     CatNose     0.8604    0.5500    0.6711      1389
      DogEye     0.6945    0.7206    0.7074      2316
     DogFace     0.6813    0.7884    0.7310      1158
    DogMouth     0.5564    0.9206    0.6936      1158
     DogNose     0.7922    0.4413    0.5668      1158

    accuracy                         0.7491     12732
   macro avg     0.7627    0.7464    0.7341     12732
weighted avg     0.7812    0.7491    0.7482     12732
```

The model managed to score a validation loss score of 0.7318, this value represents the measurement of the model's prediction error using the loss function. It is preferred to have this value as low as possible.

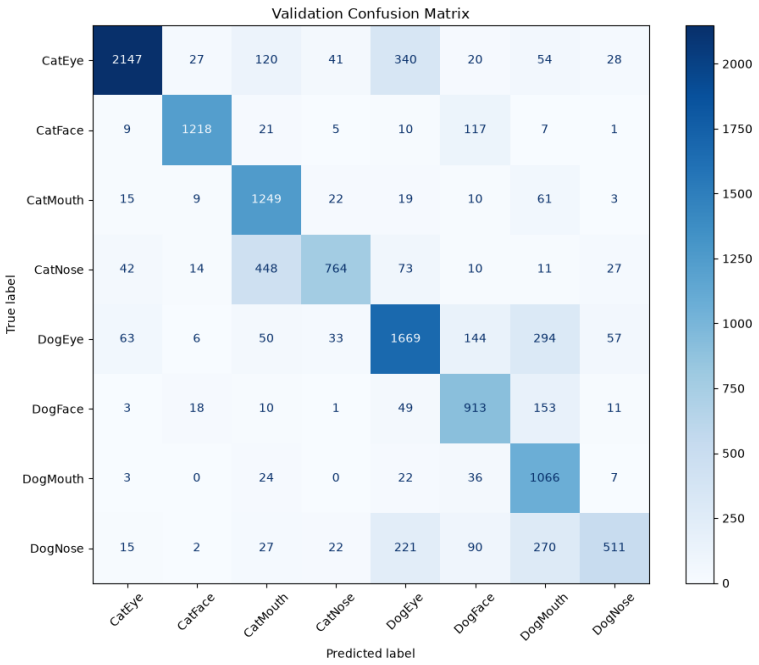
The measured accuracy comes out to ~74.82% which is mediocre at best. The scores on the Kaggle leaderboards shows figures nearing 90% to 98% accuracy so my model is still far from scoring a good score. This metric measures the total portion of images correctly classified.

Precision came out to be 78.12% which is again decent but is not what is the most relevant to the purpose of this project. I believe it would be most beneficial if I could sacrifice this score for a much higher accuracy score. To recap, the precision metric measures the proportion of correct predictions in a class, that actually were of that class.

Recall scored again a relatively good score at 74.91%; it measures the total portion of a class that was correctly identified. Again, this metric while scoring relatively well is not the goal of the model and should be sacrificed for a higher accuracy metric if possible.

F1 scored 74.82% which is again relatively good but not the goal, it combines recall and precision into a single metric which are both metrics useless for this project.

These metrics I took primarily of the weighted average as this variant of the metrics averages the class metrics while also accounting for the number of validation image cases in each class. This generates a slightly more fair metric when classes are imbalanced in representation (which applies to the classes present in this project).



The class with the 'best' performance was CatFace. It has both high recall and high precision as we can tell by the diagonal value divided by either the row sum or column sum.

The 'worst' performing class was DogNose which has a rather low recall value as well as gets misclassified a lot.

However, 'best' and 'worst' depends on the definition, and for the above I used the general across the board metrics to define them as best or worst. For this project, 'best' would mean highest accuracy, so we can ignore the above.

Accuracy can be calculated by summing up the diagonal tiles then divided by the total number of samples, which corresponds to the accuracy metric we generated above in the classification report.

Overall, other metrics remain good across the board, which put bluntly is a problem for this model simply because the goal is not to have good across the board metrics in all measures but rather to specialize and excel is just one metric, accuracy.

So, moving forward it is optimal for me to discard all other metrics and funnel the model into generating the highest accuracy metric I can possibly get it to have.

Another thing to mention, the score I received on Kaggle generally reflects the accuracy of my model to a good degree. However, it should be noted that different databases and image sets introduces variance always. The metrics the model scores on this training image set may not always mean it will translate well into another image set. Generalisation techniques like horizontal flipping and slightly rotating the images helps but cannot provide the guarantee that the model will do well in all image sets due to how fundamentally the machine learns.

Using the Kaggle submission as an example, perhaps there is a little variation between the test image set and the training image set which will cause a mismatch of metrics. This is simply a theory, the differences between the Kaggle image set and the training image set has not been investigated and so should be taken with a grain of salt.

Task 6:

From what I can gather from the above statistics and metrics:

- Validation accuracy – 74.91%
- Kaggle score – 68.42%
- Validation loss – 0.7318

The numbers tells us that the model correctly classifies approximately ¾ images into their corresponding classes. This is a useful as a baseline, but the 25.09% error rate is a little concerning, furthermore without optimizations it does not come close to the average score displayed by the Kaggle leaderboards. I would hope to come closer to a score around 95% if I can.

With the generated validation accuracy of 74.91% given the 29,443 images in the test image set we can roughly expect 22,055 images to be correctly classified into their respective classes, it's decent but far from impressive nor useful enough.

However, it has been shown that this is not the case. The Kaggle score of 68.42% proves this as they do not match. I suspect the difference points at the model's generalisation not being relatively perfected yet. Same model, same checkpoint, different image set, different score.

Lastly, the validation loss of 0.7318 acts as the measure of the model's prediction loss under our weighted cross-entropy objective (meaning it is with the class weights included). I should aim to lower this score if possible.

	Class	Correct predictions	Total validation images	Class accuracy (%)
0	CatEye	2147	2777	77.31
1	CatFace	1218	1388	87.75
2	CatMouth	1249	1388	89.99
3	CatNose	764	1389	55.00
4	DogEye	1669	2316	72.06
5	DogFace	913	1158	78.84
6	DogMouth	1066	1158	92.06
7	DogNose	511	1158	44.13

The individual class evaluation reveals variation in the CNN's classification performance for each class. CatFace achieved the highest F1-score of 90.83%, and a precision of 94.13% and recall of 87.75%. This suggests the model can distinguish cat-face images relatively reliably. In contrast, DogNose achieved the lowest F1-score of 56.68% and recall of 44.13%, making it the most difficult class according to these measures. Checking the table and yes, it indeed has the lowest class accuracy of all 8 target classes.

CatNose also demonstrated relatively low recall at 55.00%. These metrics indicates that a large proportion of nose images are being assigned to other classes. Conversely, CatMouth and DogMouth achieved recall values of 89.99% and 92.06%, respectively, but had lower precision. DogMouth's precision was 55.64%, suggesting that many predictions assigned to this class were incorrect.

As we can tell by the generated table, the model has an easier time correctly classifying certain images than others across all 8 target classes. For example, DogMouth with a 92.06% class accuracy meets a high number of correct classifications, compared to DogNose with the lowest at 44.13% it has more than double the number of correct classifications. The model's most difficult classifications were DogNose (44.13%) and CatNose (55.00%), the rest generally got ¾ classifications correct. As for why both difficult class were of Noses I have no current theory nor evidence.

As for limitations on the performance of the model, I believe it to be the mix of all factors. For example, a not large enough image size, or a not as powerful optimizer, perhaps the generalisation wasn't as effective as I had thought?

Side note, I had expected CatMouth or CatNose to have the highest class accuracy due to the presence of whiskers in a cat but was proven wrong.

To these issues I propose to change and optimize nearly every arbitrary model decision I have chosen so far using this custom CNN with ADAM as a benchmark.

For example, custom CNNs like these are flexible because they can apply to whatever you need, but they don't specialize in one area as I need. A heavier fine tuned model would suit a specific need better, so perhaps one that has been pretrained on what I need.

As for inputs certain features on pets can sometimes be a little unclear, perhaps a larger image size could convey finer details than my 128px could.

Again, a pretrained optimizer would potentially perform better as they would excel in exactly what I need.

Augmentations could be stronger and 'weirder'. Crops, flips, colour changes, and more could generalize the model much more than a simple flip and rotate could.

More epochs would be a simple way of generating a slightly better result with the same model, I could perhaps raise the number a little.

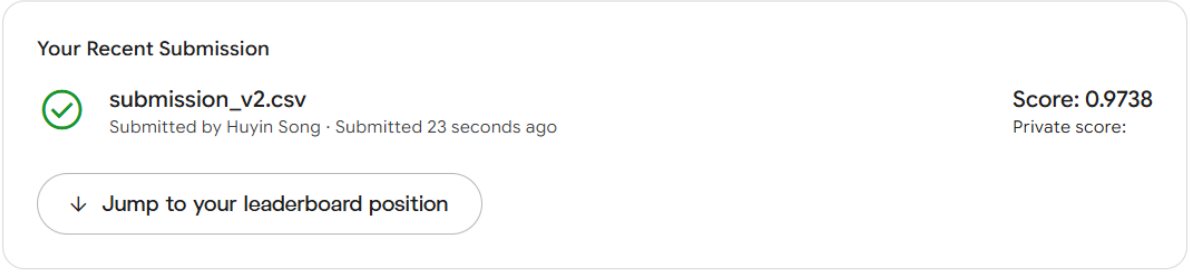
To these considerations I have decided to go with a more pretrained model that is finer tuned towards what I need exactly.

I went with a 'ConvNeXt' model that was pretrained on already pretrained images (allowed by project rules) and resized images to 160 x 160px. Next, I used a AdamW optimizer with a weight decay of 0.05 hoping it would generate a better result.

As for augmentations; crop has been added as well as flips as usual and a little colour changing in random.

The number of epochs has also been increased from 10 to 15 passes, which on my RTX 3070 took roughly 32 minutes but will provide slightly better checkpoints for accuracy.

The 80/20 split can stay; it makes sense for the application as we're focusing on as much data as we can without messing up validation.



As the result my model has benchmarked a relatively high Kaggle competition score, I attribute this advantage to the pre-trained architecture and optimizer.

The strengths of this new v2 model is how it can benefit from previously pre-trained patterns and visual features, it's like having more data to learn from. The higher image quality as well as stronger augmentations enables a higher generalization enabling a good transition from training and evaluation to application.

As for weaknesses, the v2 model definitely eats more resources and memory. Sometimes random cropping and colour adjustments may also change useful features which can hurt performance.

Task 7:

Huyin Song (25331396) ENGE707 Kaggle AI Competition Assessment

Kaggle username: huyinsong (Huyin Song)

